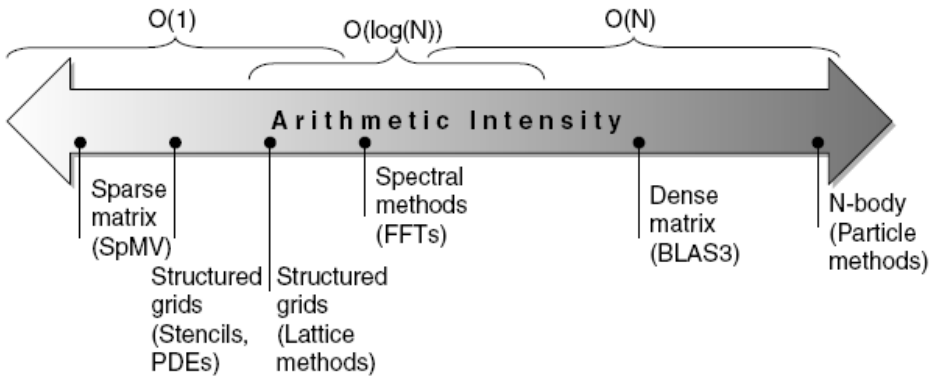
Roofline Performance Model

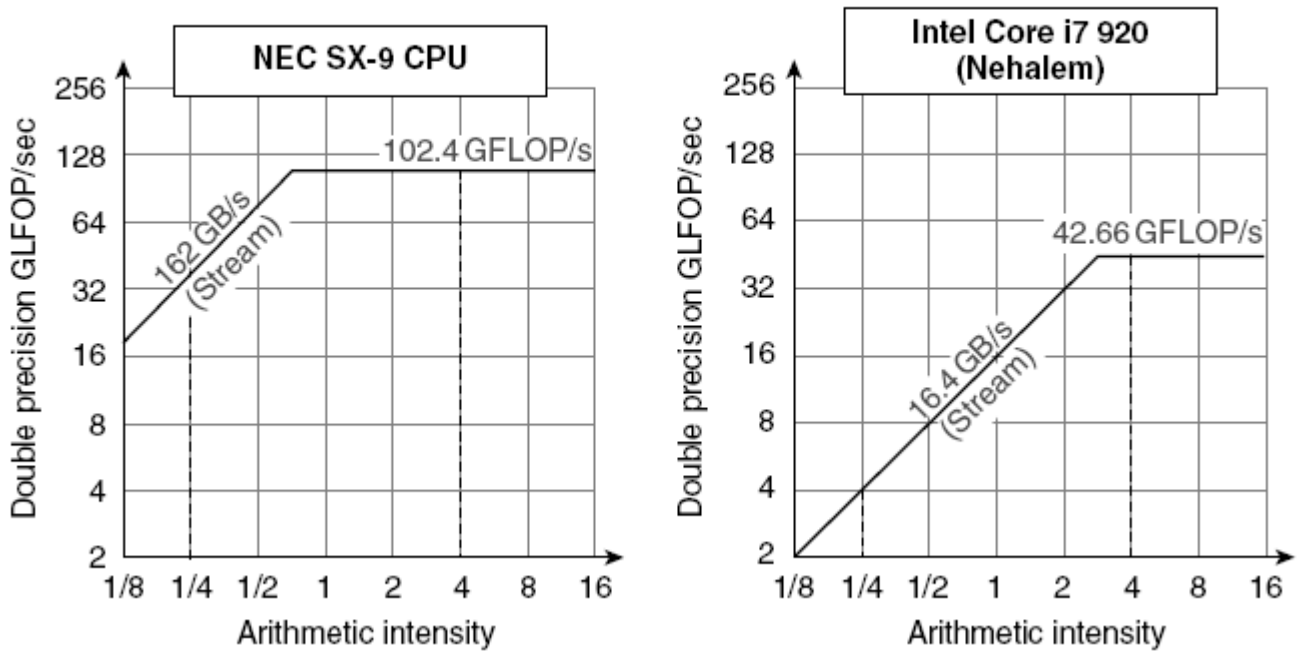
- Basic idea:
  - Plot peak floating-point throughput as a function of arithmetic intensity
  - Ties together floating-point performance and memory performance for a target machine
- Arithmetic intensity
  - Floating-point operations per byte read (and write, depending on which is more dominant)



Copyright © 2012, Elsevier Inc. All rights reserved.

Examples

- Attainable GFLOPs/sec = Min (Peak Memory BW × Arithmetic Intensity, Peak Floating Point Perf.)



Copyright © 2012, Elsevier Inc. All rights reserved.

Examples: NVIDIA GPUs

Using the roofline model: matrix multiplication

```
void MatrixMulOnHost(const Matrix M, const Matrix N, Matrix P)
{
    for (int i = 0; i < M.height; ++i)
        for (int j = 0; j < N.width; ++j) {
            double sum = 0;
            for (int k = 0; k < M.width; ++k) {
                double a = M.elements[i * M.width + k];
                double b = N.elements[k * N.width + j];
                sum += a * b;
            }
            P.elements[i * N.width + j] = sum;
        }
}
```

- Arithmetic Intensity = ?
- BW bound or Computational Bound for GTX980 GPU, GTX3080 GPU?

Using the roofline model: Vector Add

```
for (int i = 0; i < N; i++)
    S[i] = A[i]+B[i];
```

A, B single precision FP.

- Arithmetic Intensity = ?
- BW bound or Computational Bound for GTX980 GPU, GTX3080GPU?

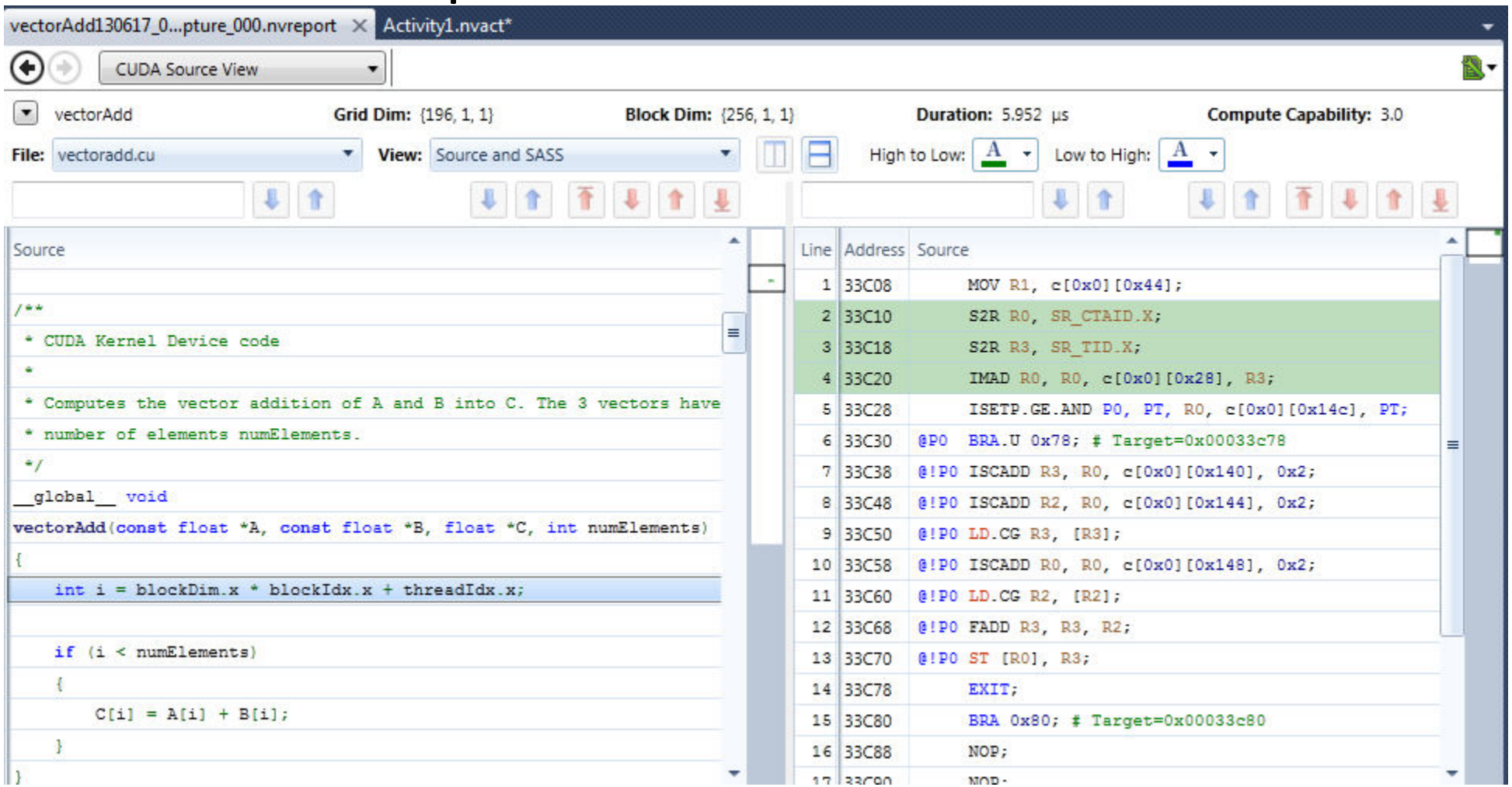
NVIDIA Instruction Set Arch.

- ISA is an abstraction of the hardware instruction set
  - “Parallel Thread Execution (PTX)”
  - Uses virtual registers
  - Translation to machine code is performed in software
  - Example:

```
shl.s32         R8, blockIdx, 9      ; Thread Block ID * Block size (512 or 2^9)
add.s32         R8, R8, threadIdx    ; R8 = i = my CUDA thread ID
ld.global.f64   RD0, [X+R8]          ; RD0 = X[i]
ld.global.f64   RD2, [Y+R8]          ; RD2 = Y[i]
mul.f64 RD0, RD0, RD4   ; Product in RD0 = RD0 * RD4 (scalar a)
add.f64 RD0, RD0, RD2   ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0            ; Y[i] = X[i]*a + Y[i]
```

Copyright © 2012, Elsevier Inc. All rights reserved.

Native ISA: Example



Three approaches to handle conditional branches in GPUs

- SIMT stack based approach (i.e., PDOM)
- Predication
- Predication + Unanimous Branches

Control Instruction in NVIDIA GPUs

Mnemonic	**Description**	**Semantic**
SSY	Set Synchronization	SSY 0xf0;
BRA	Direct Branch	@P0 BRA 0xb8;
BRA.U	Uniform branch	@P0 BRA.U 0xb8;
BRX	Indirect Branch	BRX R2 0x0;
PRET	Used with BRX	PRET 0x60;
CALL	function call	CAL 0x168;
RET	Returning from function call	RET;
PBRK	Prebreak	PBRK 0x180;
BRK	Break	BRK or @P0 BRK;
".S" flag	Stack Pop operation	

Hardware Support for Control Instructions

ACTIVE MASK	RECONVERGENCE PC	ENTRY TYPE

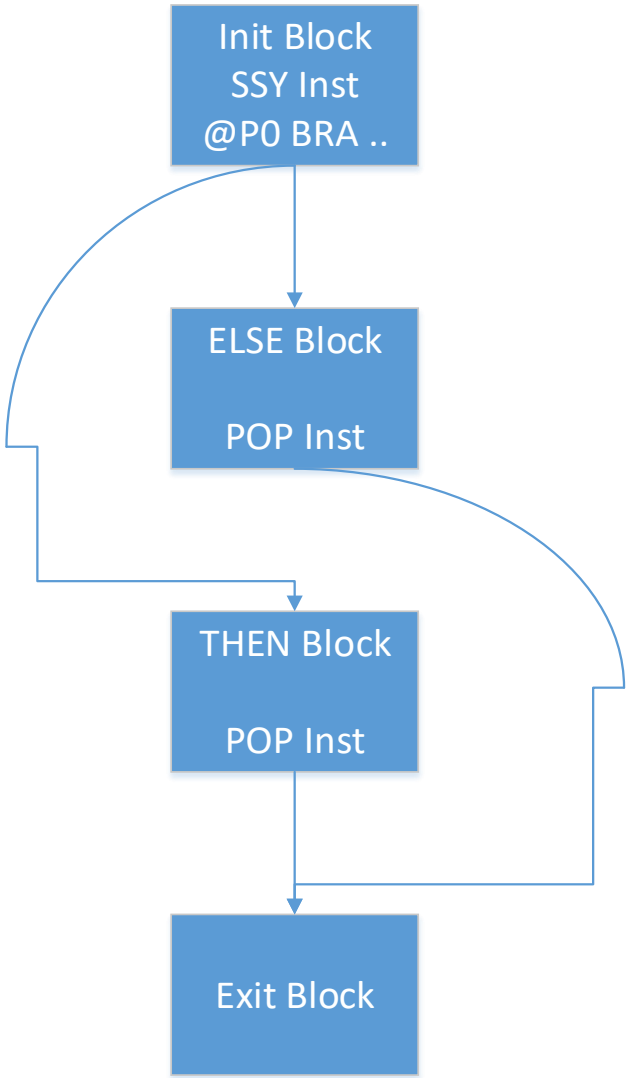
Instruction	Divergent?	Token Type	Active Mask	Reconvergence PC
SSY	------	SSY	Active Mask	Sync PC
BRA	N	------	------	------
BRA	Y	DIV	Taken Mask	Taken PC
PRET	------	PRET	Active Mask	Sync PC
BRX	N	-----	------	------
BRX	Y	DIV	Non taken Mask	Active PC
CALL	------	CALL	Active Mask	Non Taken PC
PREBRK	------	PBK	Active Mask	Sync PC

Reference: Coon et al. United States Patent US007761697Bl: PROCESSING AN INDIRECT BRANCH INSTRUCTION IN A SIMD ARCHITECTURE. Assignee NVIDIA Corporation, Santa Clara, CA (US)

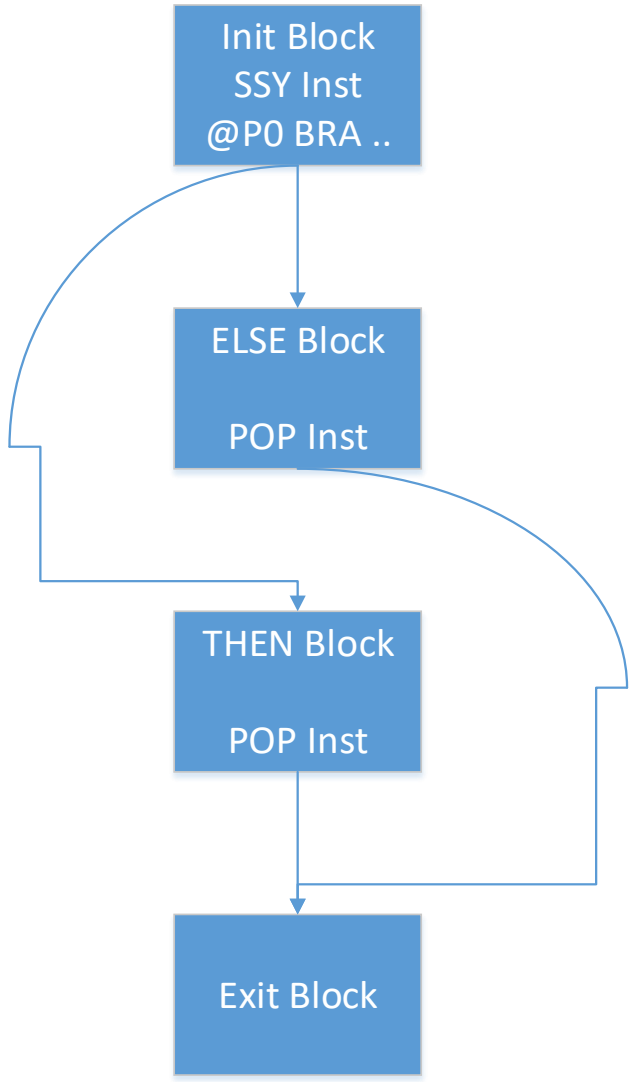
Forward Branch Stack based Implementation

- IF-Then Else, SWITCH Case Structure Implementation
- Requires stack in this implementation

```
int index = blockIdx.x*blockDim.x +
threadIdx.x;
      if (index > x1) // Section 1 - THEN
      {
            …
       }
       Else          //Section 1- ELSE
       {
           …
       }
d_e [index] = d_c [index] + d_d [index] ;
```



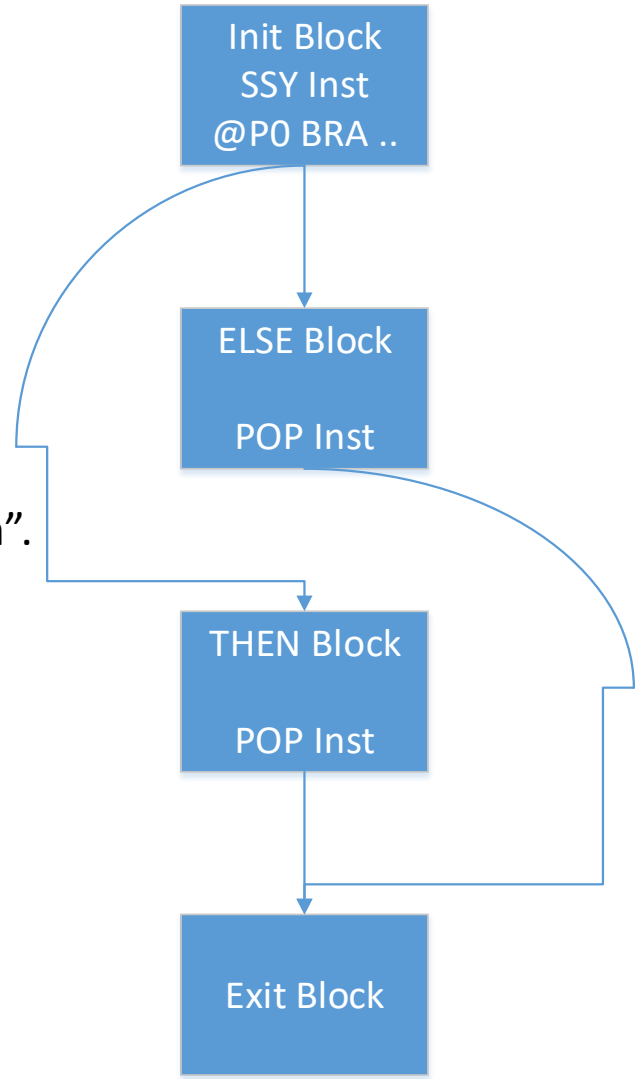
PC	Instruction	Comment
0000	MOV R1, c [0x1] [0x100];	
0020	SSY 0xf0;	SSY Instruction (push stack)
0028	…..	…..
0090	@P0 BRA 0xb8;	Branch corresponding to Outer IF Then-Else (push stack)
0098	LD.E R3, [R2];	ELSE PART
….		
00b0	ST.E.S [R6], R2;	ELSE PART. Notice ".S" flag. (pop stack)
00b8	LD.E R5, [R4];	IF THEN PART
…	….	….
00e8	NOP.S CC.T;	Last Instruction of Outer If-Then. Notice ".S" flag. (pop stack)
00f0	LD.E R3, [R10];	Threads Synchronizes at this point.
00f8	LD.E R4, [R8];	….
0100	LD.E R2, [R6];	Go till Exit.
….	….	….
0140	EXIT;	



PC	Active Mask	TOS	TOS-1	TOS-2	Comments
0	11111111	Empty	Empty	Empty	
20	11111111	Empty	Empty	Empty	Before SSY
28	11111111	SSY, 0xFF, 0xF0	Empty	Empty	After SSY
90	11111111	SSY, 0xFF, 0xF0	Empty	Empty	Before Branch
98	00001111	DIV, 0xF0, 0xB8	SSY, 0xFF, 0xF0	Empty	After Branch
…	00001111	DIV, 0xF0, 0xB8	SSY, 0xFF, 0xF0	Empty	
B0	00001111	DIV, 0xF0, 0xB8	SSY, 0xFF, 0xF0	Empty	Before ST.S
B8	11110000	SSY, 0xFF, 0xF0	Empty	Empty	After ST.S
…	11110000	SSY, 0xFF, 0xF0	Empty	Empty	
…	11110000	SSY, 0xFF, 0xF0	Empty	Empty	
0xE0	11110000	SSY, 0xFF, 0xF0	Empty	Empty	
0xE8	11110000	SSY, 0xFF, 0xF0	Empty	Empty	“NOP.S Instruction”.
0xF0	11111111	Empty	Empty	Empty	
0xF8	11111111	Empty	Empty	Empty	Go Till Exit.

PC	Active Mask	TOS	TOS-1	TOS-2	Comments
0	11111111	Empty	Empty	Empty	
20	11111111	Empty	Empty	Empty	Before SSY
28	11111111	SSY, 0xFF, 0xF0	Empty	Empty	After SSY
90	11111111	SSY, 0xFF, 0xF0	Empty	Empty	Before Branch
98	11111111	SSY, 0xFF, 0xF0	Empty	Empty	After Branch
…	11111111	SSY, 0xFF, 0xF0	Empty	Empty	
B0	11111111	SSY, 0xFF, 0xF0	Empty	Empty	Before ST.S
F0	11111111	Empty	Empty	Empty	After ST.S

Go Till Exit.



Predication

```
if (threadIdx.y<2) {
    i = threadIdx.x;
    j = threadIdx.y*(BLOCK_Y+1) - 1;
      if (A) {
      }
  }
  else {
    i = (k%2)*(BLOCK_X+1) - 1;
    j =  k/2 - BLOCK_X - 1;
  }
```

```
                    P0 = (threadidx < 2)
/*0060*/           @!P0 MOV32I R6, 0x1;
/*0068*/           @!P0 MOV R2, R0;
/*0070*/           @!P0 IMAD.U32.U32 R3, R3, 0x5, -R6;
/*0078*/           @P0 SHR.W R3, R2, 0x1f;
/*0080*/           @P0 MOV32I R6, 0x1;
/*0088*/           @P0 LOP.AND R3, R3, 0x1;
/*0090*/           @P0 IADD R3, R3, R2;
/*0098*/           @P0 SHR.W R3, R3, 0x1;
/*00a0*/           @P0 ISCADD R2, -R3, R2, 0x1;
/*00a8*/           @P0 IADD R3, R3, 0xfffdf;
/*00b0*/           @P0 IMAD R2, R2, 0x21, -R6;
```

Nested branches
Predication:
Setp p0
!P0 setp p1 = A
P1
!p1 & !p0
p0

unanimous branch instruction BRA.U

- Semantics of BRA.U: if there is no divergence among the threads in a warp, the branch is taken. Otherwise, the branch is not taken.

Nickolls et al. United States Patent 2011/0072249: UNANIMOUS BRANCH INSTRUCTIONS IN A PARALLEL THREAD PROCESSOR (Assignee NVIDIA Corp.), March 2011

Forward branch “BRA.UNI” based Implementation

```
//TB dimension: 8x32; warp size: 32
//if(threadIdx.y%2 == 0)
if (threadIdx.y<2) {
     i = threadIdx.x;
     j = threadIdx.y*(BLOCK_Y+1) - 1;
   }
   else {
     i = (k%2)*(BLOCK_X+1) - 1;
     j =  k/2 - BLOCK_X - 1;
   }
```

```
/*0058*/          @P0 BRA.U 0x80;
/*0060*/          @!P0 MOV32I R6, 0x1;
/*0068*/          @!P0 MOV R2, R0;
/*0070*/          @!P0 IMAD.U32.U32 R3, R3, 0x5, -R6;
/*0078*/          @!P0 BRA.U 0xc0;
/*0080*/          @P0 SHR.W R3, R2, 0x1f;
/*0088*/          @P0 MOV32I R6, 0x1;
/*0090*/          @P0 LOP.AND R3, R3, 0x1;
/*0098*/          @P0 IADD R3, R3, R2;
/*00a0*/          @P0 SHR.W R3, R3, 0x1;
/*00a8*/          @P0 ISCADD R2, -R3, R2, 0x1;
/*00b0*/          @P0 IADD R3, R3, 0xfffdf;
/*00b8*/          @P0 IMAD R2, R2, 0x21, -R6;
```

Case 1 Divergence :

PC	Active Mask	Active Threads
58	11111111	11111111
60	11111111	11110000
68	11111111	11110000
70	11111111	11110000
78	11111111	11110000
80	11111111	00001111
88	11111111	00001111
….		
B8	11111111	00001111
C0	11111111	11111111

…Till Exit

Case 2 No Divergence (All threads Non-Taken) :

PC	Active Mask	Active Threads
58	11111111	11111111
60	11111111	11111111
68	11111111	11111111
70	11111111	11111111
78	11111111	11111111
C0	11111111	11111111

…Till Exit

Case 3 No Divergence (All threads Taken):

PC	Active Mask	Active Threads
58	11111111	11111111
80	11111111	11111111
88	11111111	11111111
…	11111111	11111111
C0	11111111	11111111

…Till Exit

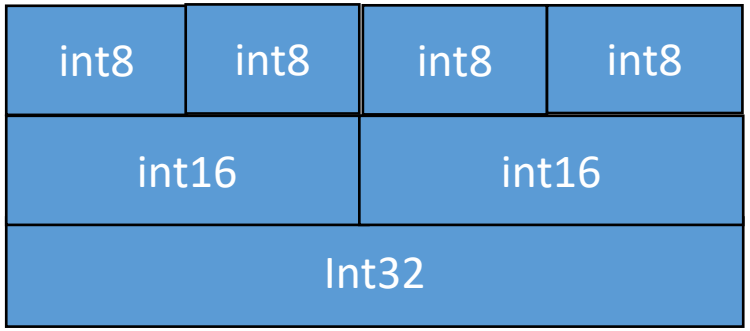
```
/*0058*/        @P0 BRA.U 0x80;
/*0060*/        @!P0 MOV32I R6, 0x1;
/*0068*/        @!P0 MOV R2, R0;
/*0070*/        @!P0 IMAD.U32.U32 R3, R3, 0x5, -R6;
/*0078*/        @!P0 BRA.U 0xc0;
/*0080*/        @P0 SHR.W R3, R2, 0x1f;
/*0088*/        @P0 MOV32I R6, 0x1;
/*0090*/        @P0 LOP.AND R3, R3, 0x1;
/*0098*/        @P0 IADD R3, R3, R2;
/*00a0*/        @P0 SHR.W R3, R3, 0x1;
/*00a8*/        @P0 ISCADD R2, -R3, R2, 0x1;
/*00b0*/        @P0 IADD R3, R3, 0xfffdf;
/*00b8*/        @P0 IMAD R2, R2, 0x21, -R6;
```

The tradeoffs among the three approaches

- SIMT Stack (BRA):
  - Doesn’t require access to predicate register for all instructions of IF-THEN ELSE path (saves energy)
  - Used for longer branches (large instruction count).
  - No need to execute both TAKEN and NOT-TAKEN paths for unanimous branches.
- Predication
  - Eliminates stack accesses and synchronization instructions
  - Always execute both TAKEN and NOT-TAKEN paths
- Predication + BRA.UNI:
  - Eliminates stack accesses and synchronization instructions
  - No need to execute both paths for unanimous branches.
  - Used for shorter IF-THEN ELSE branches similar to predication

GPU ISA Features (ARM, AMD, NVIDIA)

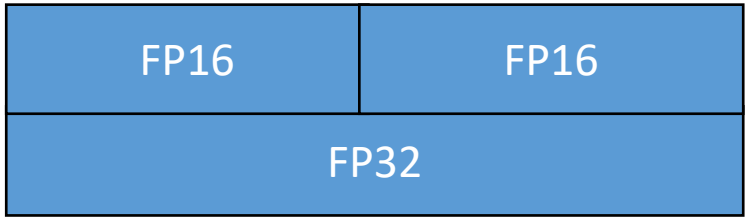
- SIMD: Support smaller-width data types
  - 2x (4x) performance for FP16 (Int8)



8-bit integer

16-bit integer

32-bit integer



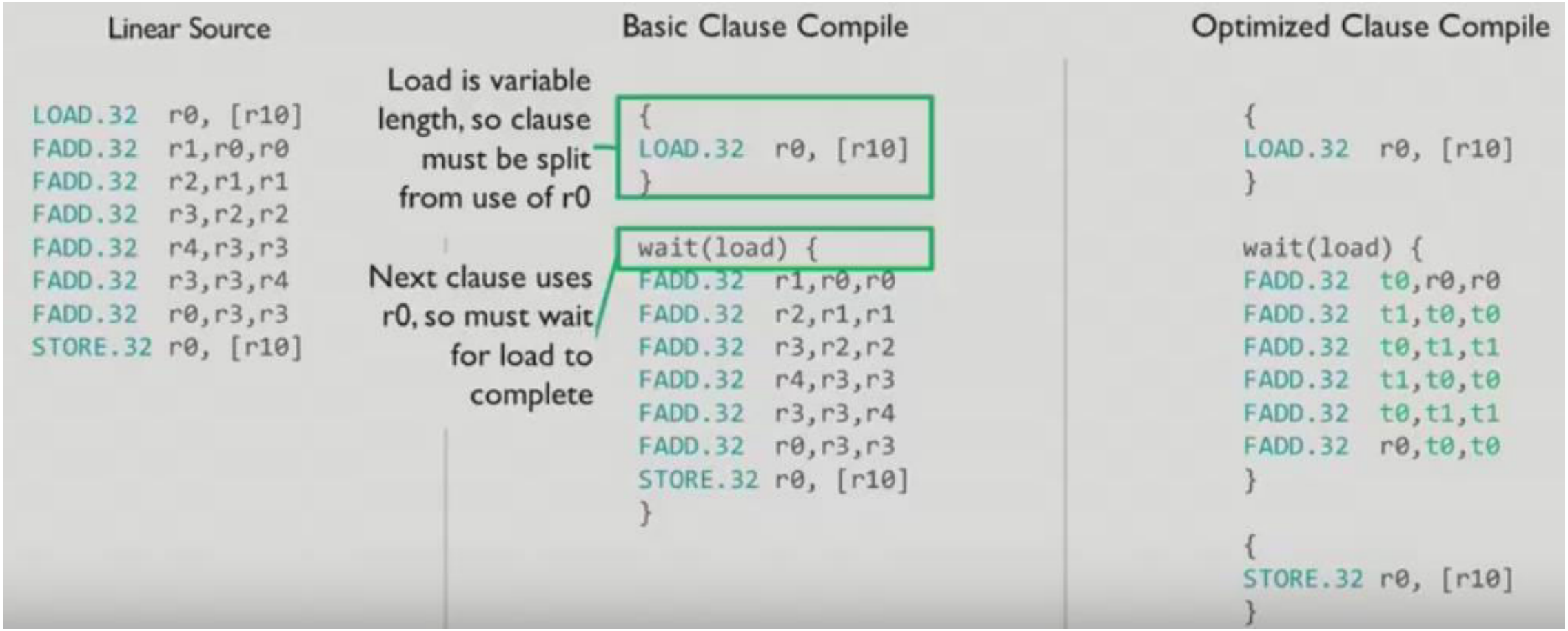
16-bit floating point

32-bit floating point

GPU ISA Features (ARM, AMD)

- Clause execution
  - A group of instructions that execute atomically: architectural states visible after clause execution.
  - Bypass path registers exposed to the compiler
  - Variable latency instructions on clause boundary

Clause example



From ARM presentation @HOTCHIPS 2016

WAITCNT insn

- Compiler schedules the instructions such that the hardware does not need to check dependency
  - Easy for fixed-latency instructions. NOPs may be inserted.
  - How about variable-latency instructions?
    - WaitCNT
    - E.g., s_waitcnt vmcnt(1)
    - VM_CNT – Vector memory count (from AMD GCN ISA)
      - Determines when memory reads have returned data to VGPRs, or memory writes have completed.

        – Incremented every time a vector-memory read or write (MIMG, MUBUF, or MTBUF format) instruction is issued.

        – Decremented for reads when the data has been written back to the VGPRs, and for writes when the data has been written to the L2 cache.

        – Ordering: Memory reads and writes return in the order they were issued, including mixing reads and writes. (Instructions of a given type return in order, but instructions of different types can complete out-of-order. The exception is scalar-memory-reads, which can return out-of-order, in which case only S_WAITCNT 0 is the only legitimate value).

Tensor Core Instructions (Nvidia)

- **Warp Level Matrix Multiply-Accumulate Instructions**
  - **D = A * B + C A,B,C,and D are matrices**
  - The shapes of all three matrix operands are collectively described by the tuple `MxNxK`, where A is an `MxK` matrix, B is a `KxN` matrix, while C and D are `MxN` matrices.

- wmma.load loads A, B, and C based on different M,N,K and precision
- wmma.mma performs A*B + C on tensor cores
- wmma.store stores D to memory based on M, N and precision